

Dijkstra

The Dijkstra algorithm is a graph algorithm used to find the shortest path from a single source vertex to all other **vertices** in a weighted graph with non-negative edge weights. It works by iteratively selecting the vertex with the smallest tentative distance, updating the distances of its neighbors, and marking it as “visited” once the shortest path to that vertex is determined. This process continues until all vertices have been processed. Dijkstra’s algorithm uses a priority queue (or min-heap) for efficient selection of the next vertex with the minimum distance, which significantly improves performance on large graphs. The algorithm is widely used in network routing, GPS navigation, and mapping applications, where shortest path calculations are crucial. Its time complexity depends on the data structure used: with a simple array, it is $O(V^2)$, while using a min-priority queue (e.g., a binary heap) it becomes $O((V + E) \log V)$, where V is the number of vertices and E is the number of edges. Dijkstra’s algorithm is efficient, easy to implement, and guarantees the shortest path for graphs without negative edge weights.

Advantages / Benefits

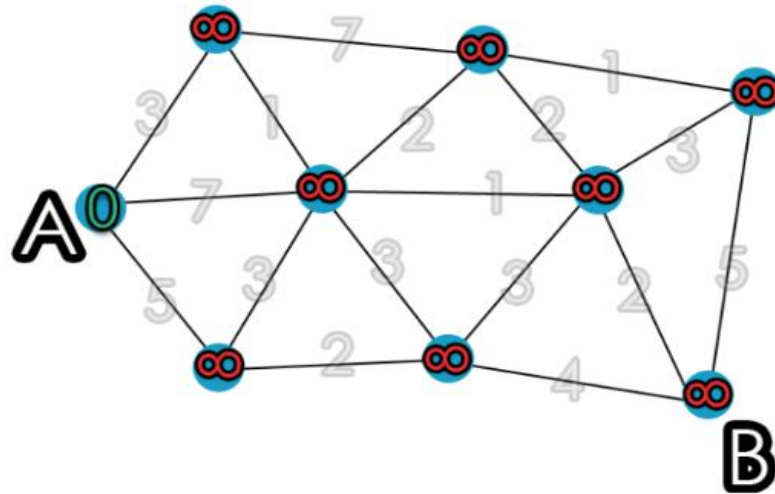
- Efficiently finds the shortest path from a single source to all vertices.
- Guarantees optimal paths in graphs with non-negative edge weights.
- Widely used in network routing, GPS navigation, and mapping applications.
- Can be efficiently implemented using a priority queue or min-heap for large graphs.
- Works for both dense and sparse graphs when appropriate data structures are used.
- Once a vertex is processed, its shortest distance is final, simplifying path reconstruction.

Limitations

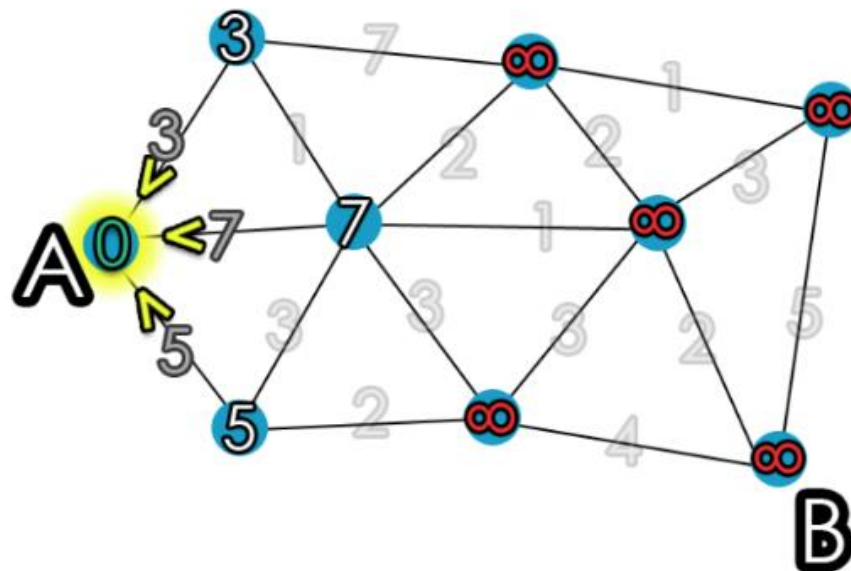
- Cannot handle negative edge weights, unlike Bellman-Ford.
- Time complexity with a simple array is $O(V^2)$, which can be inefficient for large graphs.
- Even with a priority queue, time complexity is $O((V + E) \log V)$, which may be slow in some cases.
- Does not detect negative weight cycles, so it is unsuitable for graphs containing them.
- For dynamic graphs where edges change frequently, the algorithm may need to be rerun entirely.

Steps:

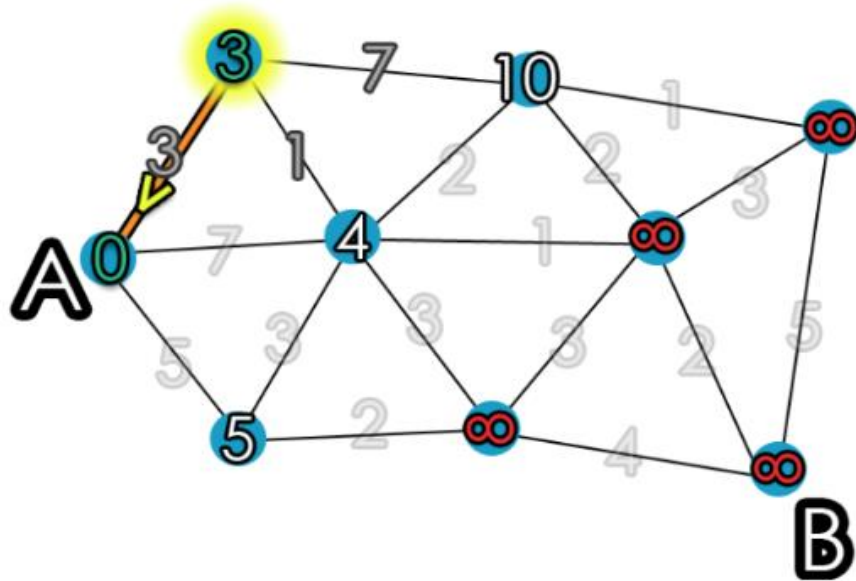
1. Initialize distances according to the algorithm.



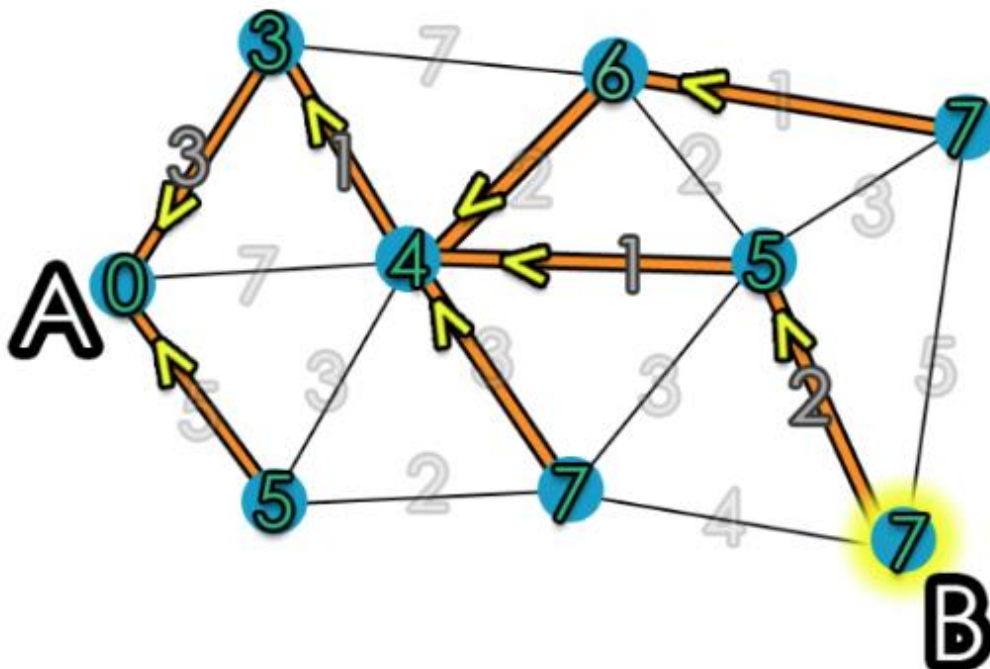
2. Pick first node and calculate distances to adjacent nodes.



3. Pick next node with minimal distance; repeat adjacent node distance calculations.



4. Final result of shortest-path tree



Pseudocode:

DIJKSTRA(source, n):

 INITIALIZE-SINGLE-SOURCE(source, n)

 CREATE priority_queue pq (min-heap using negative values)

 pq.push({0, source})

 WHILE pq is not empty:

 node = pq.top()

 pq.pop()

 u = node.second

 FOR EACH neighbor v of u:

 w = weight(u, v)

 // Relaxation step

 IF $\text{dis}[u] + w < \text{dis}[v]$:

$\text{dis}[v] = \text{dis}[u] + w$

 parent[v] = u

 pq.push({-dis[v], v})

INITIALIZE-SINGLE-SOURCE(source, n):

 FOR i = 0 TO n-1:

$\text{dis}[i] = \infty$

 parent[i] = -1

 dis[source] = 0

Time Complexity:

Time Complexity: $O((V + E) \log V)$

Where:

V = Number of Vertices/Nodes

E = Number of Edges

Breakdown:

Operation	Count	Complexity per Operation	Total
Initialization	V	$O(1)$	$O(V)$
Push (insert)	V	$O(\log V)$	$O(V \log V)$
Pop (extract-min)	V	$O(\log V)$	$O(V \log V)$
Push (decrease-key)	E	$O(\log V)$	$O(E \log V)$

Total: $O((V + E) \log V)$

Dijkstra Code:

<https://github.com/AHANIK33/Algorithm/blob/main/Dijkstra/Basic/basic.cpp>